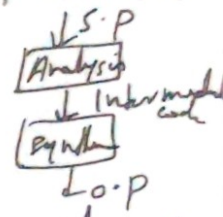
Module 4

Intermediate Code Generation

* In the analysis - synthesis model of a compiler, the front end translates a source program into an intermediate representation from which the back end generates target code.



Benefits

* Retargeting is facilitated, a compiler for a different machine can be created by attaching a back end for the new machine to an existing front end.
* A machine independent code optimizer can be applied to the intermediate representation.

Intermediate Languages

* Syntax trees, postfix notation and three address codes are three kinds of intermediate representation
* syntax trees - It is a condensed form of parse tree useful for representing language constructs.
* In syntax trees, operators do not appear as leaves, but are associated with the interior nodes.

Consider,

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow digit$

Given string is, $3*5+4$

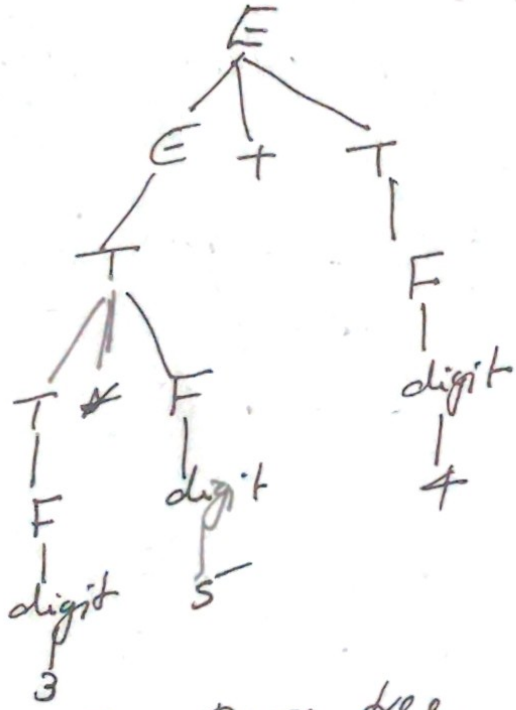


Fig parse tree

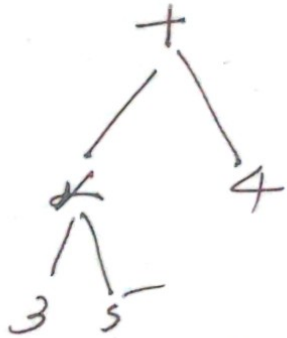
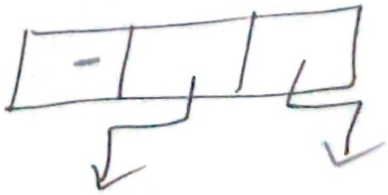


Fig. Syntax tree.

* Constructing syntax trees for expressions
* Each node in a syntax tree can be implemented as a record with several fields.
* In the node for an operator, one field identifies the operator and the remaining fields contain pointers to the nodes for the operands.



* The following functions are used to create the nodes of syntax trees for expressions with binary operators.

mknode (op, left, right) creates an operator node with label op and two fields containing pointers to left and right.

mkleaf (id, entry) creates an identifier node with label id and a field containing entry, a pointer to the symbol table entry for the identifier.

mkleaf (num, val) creates a number node with label num and a field containing val, the value of the number.

* The following sequence of function calls creates the syntax tree for the expression a - 4 + c as in fig. In this sequence p1, p2, . . . p5 are pointers to nodes and entrya and entryc are pointers to the symbol table entries for identifiers a and c respectively.

(1) p1 = mkleaf (id, entrya)

(2) p2 = mkleaf (num, 4)

(3) p3 = mknode ('-', p1, p2)

(4) p4 = mkleaf (id, entryc)

(5) p5 = mknode ('+', p3, p4)

The tree is constructed bottom-up. The function calls mkleaf (id, entrya) and mkleaf (num, 4) construct the leaves for a and 4. The pointers to these nodes are saved using p1 and p2. The call mknode ('-', p1, p2) then constructs the interior node with the leaves for a and 4 as children.

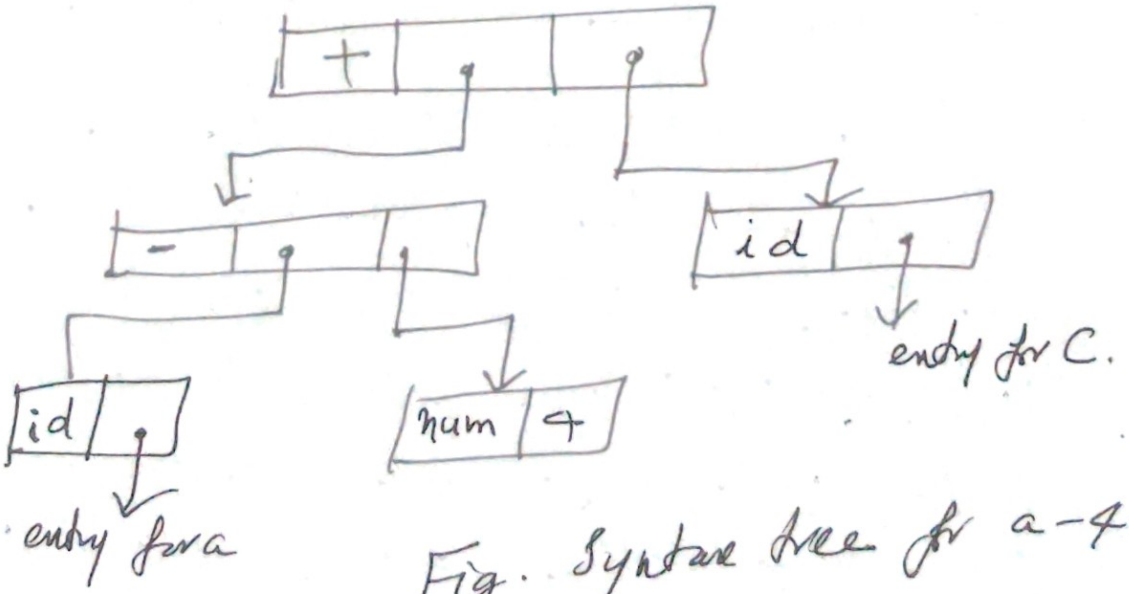
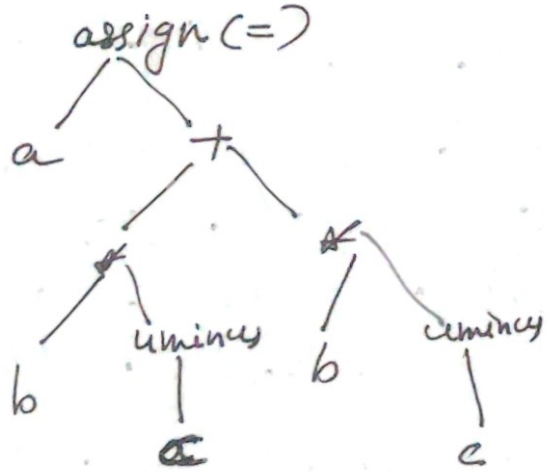


Fig. Syntax tree for a-4*c

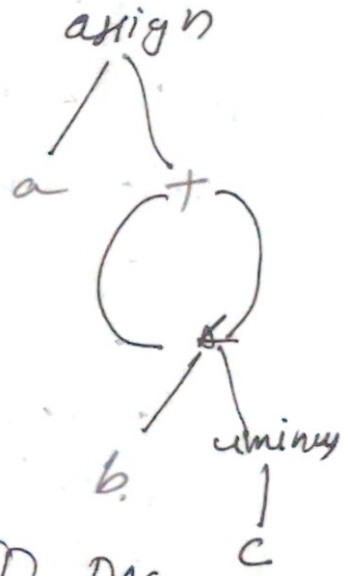
Directed acyclic graphs (DAG) for expressions

* A DAG for an expression identifies the common subexpressions in the expression.
* A node in a DAG representing a common subexpression has more than one parent in a syntax tree, the common subexpression ~~has more than one parent~~ would be represented as a duplicated subtree.

→ Example, $a = b * -c + b * -c$



syntax tree



(b) DAG.

→ $a + a * (b-c) + (b-c) * d$

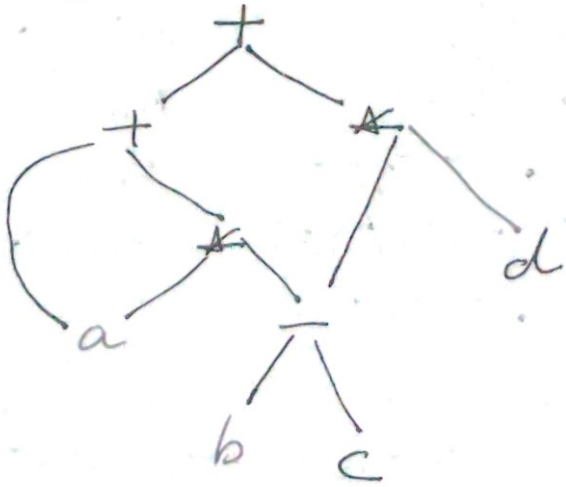


Fig. DAG for the expression $a+a*(b-c)+(b-c)*d$

* Here, the leaf for a has two parents because 'a' is common to the two subexpressions a and a * (b−c).
* Likewise, both occurrances of the common subexpression b−c are represented by the same node, which also has two parents..
* A DAG is ~~cons~~ obtained if the function constructing a node first checks to see whether an identical node already exists.
* For example, before constructing a new node with label op and fields with pointers to left and right, mknode (op, left, right) can check whether such a node has already been constructed. If so mknode (op, left, right) can return a pointer to the previously constructed node..

* p1 = mkleaf (id, a)
p2 = mkleaf (id, a)
p3 = mkleaf (id, b)
p4 = mkleaf (id, c)
p5 = mknode ('−', p3, p4)
p6 = mknode ('*', p2, p5)
p7 = mknode ('+', p1, p6)
p8 = mkleaf (id, b)
p9 = mkleaf (id, c)
p10 = mknode ('−', p8, p9)
p11 = mkleaf (id, d)
p12 = mknode ('*', p10, p11)
p13 = mknode ('+', p7, p12)

Fig. Instructions for constructing DAG
a + a * (b−c) + (b−c) * d

* When the call mkleaf (id, a) is repeated on line 2, the node constructed by the previous call mkleaf (id, a) is returned, ~~so~~.
* Similarly, the nodes returned on lines 8 and 9 are the same as those returned on lines 3 and 4 respectively. Hence, the node returned on line 10 must be the same as constructed by the call of mknode on line 5.
* The syntax directed definition for the input a=b*-c+b*-c is given below.

Production	Semantic Rule
$S \rightarrow id = E$	S.nptr = mknode('assign', mkleaf(id, id.place), E.nptr)
$E \rightarrow E_1 + E_2$	E.nptr = mknode('+', E_1.nptr, E_2.nptr)
$E \rightarrow E_1 * E_2$	E.nptr = mknode('*', E_1.nptr, E_2.nptr)
$E \rightarrow -E_1$	E.nptr = mknode('uminus', E_1.nptr)
$E \rightarrow (E_1)$	E.nptr = E_1.nptr
$E \rightarrow id$	E.nptr = mkleaf(id, id.place)

* Two representations for syntax tree are given in following figs. (a) and (b). Each node is represented as a record with a field for its operator and additional fields for pointers to its children.

* In Fig (b) nodes are allocated from an array of records and the index or position of the node serves as the pointer to the node.

*

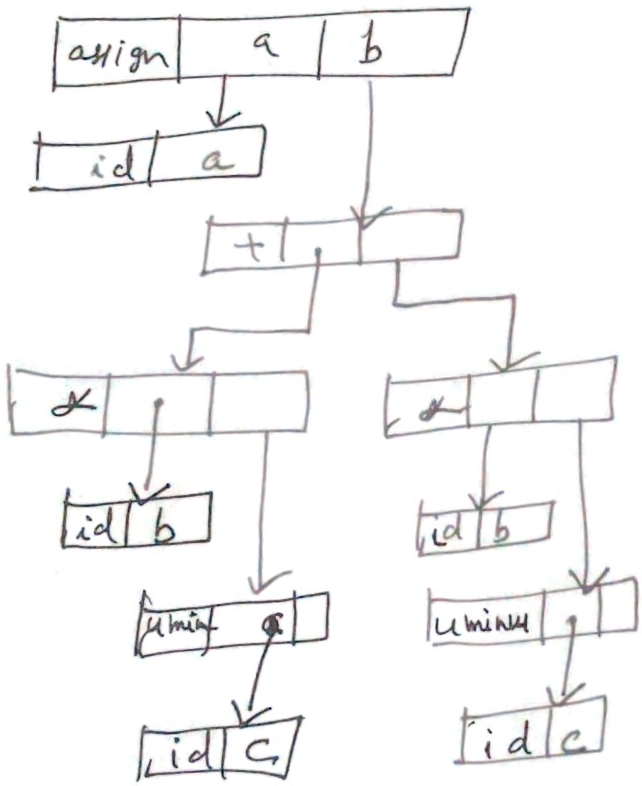


Fig (a)

0	id	b	
1	id	c	
2	uminus	1	
3	*	0	2
4	id	b	
5	id	c	
6	uminus	5	
7	*	4	6
8	+	3	7
9	id	a	
10	assign	9	8
11			

Fig. (b)

Fig. Two representations of the syntax tree for the expression

$a = b * -c + b * -c$